

State Space Models (SSM)

Detailed Notes based on Deep Learning for Time Series at Ecole Polytechnique

Contents

1	Motivation and model formulation	3
1.1	Continuous-time SSM	3
1.2	Intuition of the matrices	3
2	Comparison with RNNs, CNNs, Attention, and Neural ODEs	4
2.1	SSMs versus RNNs	4
2.2	SSMs versus CNNs	5
2.3	SSMs versus self-attention	5
2.4	SSMs versus Neural ODEs	5
3	ODE background: why matrix exponentials appear	5
3.1	Scalar homogeneous linear ODE	6
3.2	Matrix homogeneous linear ODE	6
3.3	Why does $e^{At}h_0$ solve the ODE?	7
3.4	How to form e^{At} in practice	7
4	Solving the non-homogeneous linear ODE	7
4.1	Derivation by integrating factor	8
4.2	Apply it to SSMs	8
5	Step 1: time discretization	9
6	Discrete dynamical system: deriving $\bar{A}$, $\bar{B}$, and $\bar{C}$	9
6.1	Zero-order hold assumption	9
6.2	Derivation of $\bar{B} = A^{-1}(e^{A\Delta} - I)B$	10
6.3	What is the goal after discretization?	11
7	Efficient computation: the convolutional trick	11
7.1	Interpretation of the kernel	12

7.2	Why this helps training	12
8	Why we need a smart parametrization of A	12
8.1	Eigendecomposition	13
8.2	Why eigendecomposition makes powers and exponentials easy	13
8.3	Stable complex eigenvalues	13
8.4	Memory interpretation	14
8.5	Do we specify the eigenvalues or learn them?	14
8.6	What about V and V^{-1} ?	14
9	Learning and gradients	15
9.1	Example gradients	16
10	Algorithm: training and inference for a spectral SSM layer	17
11	Mamba and selective state-space models	18
11.1	Why input-dependence matters	18
11.2	Why Mamba does not use the same convolutional trick	18
12	Summary of the full chapter	19

1 Motivation and model formulation

We are interested in sequence or time-series modeling. The input is a sequence

$$x_1, x_2, \dots, x_T,$$

where each $x_t \in \mathbb{R}^{d_{\text{in}}}$. The goal may be forecasting, classification, denoising, representation learning, or sequence-to-sequence prediction.

A model should ideally have the following properties:

- it should capture **long-range dependencies**;
- it should be efficient for long sequences, preferably $O(T)$ or close to it;
- it should be trainable in parallel on GPUs;
- it should have a stable mathematical structure.

State Space Models (SSMs) offer a way to model sequences using an internal hidden state that evolves over time. The central idea is:

Key idea. Instead of explicitly comparing every time point with every other time point, maintain a hidden state h_t that summarizes the past and evolves according to a dynamical system.

1.1 Continuous-time SSM

The classical continuous-time linear state-space model is

$$\begin{cases} \dot{h}(t) = Ah(t) + Bx(t), \\ o(t) = Ch(t) + Dx(t), \end{cases} \quad (1)$$

where

- $x(t) \in \mathbb{R}^{d_{\text{in}}}$ is the input signal;
- $h(t) \in \mathbb{R}^N$ is the latent hidden state;
- $o(t) \in \mathbb{R}^{d_{\text{out}}}$ is the output;
- $A \in \mathbb{R}^{N \times N}$ controls the evolution of memory;
- $B \in \mathbb{R}^{N \times d_{\text{in}}}$ injects input into memory;
- $C \in \mathbb{R}^{d_{\text{out}} \times N}$ reads memory into output;
- $D \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is a direct input-output skip term.

Many deep SSM layers set $D = 0$ inside the SSM block because residual connections in the neural network already provide a direct input-output path.

1.2 Intuition of the matrices

The equation

$$\dot{h}(t) = Ah(t) + Bx(t)$$

contains two effects.

First,

$$Ah(t)$$

describes how the hidden state evolves by itself. This is the memory dynamics. If there were no input, the state would still evolve according to A .

Second,

$$Bx(t)$$

describes how the current input pushes or changes the hidden state. The matrix B maps the input space into the hidden-state space:

$$B : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^N.$$

For a small time increment dt , the input contribution is approximately

$$h(t + dt) \approx h(t) + Bx(t) dt.$$

Thus B is the *input injection matrix*.

Finally,

$$Ch(t)$$

converts hidden memory into observable output.

2 Comparison with RNNs, CNNs, Attention, and Neural ODEs

2.1 SSMs versus RNNs

A recurrent neural network has the form

$$h_t = f_{\theta}(h_{t-1}, x_t). \quad (2)$$

A discretized SSM has the form

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t. \quad (3)$$

Hence, an SSM is recurrent in nature. The difference is that the transition is not an arbitrary nonlinear function; it comes from a structured dynamical system.

	RNN	SSM
Update	$h_t = f_{\theta}(h_{t-1}, x_t)$	$h_t = \bar{A}h_{t-1} + \bar{B}x_t$
Structure	arbitrary nonlinear transition	structured dynamical transition
Training	sequential by default	recurrent or convolutional form
Long memory	can suffer from vanishing gradients	controlled through spectrum of A

2.2 SSMs versus CNNs

A one-dimensional convolutional model computes outputs using local windows. For example,

$$y_t = \sum_{j=0}^{K-1} W_j x_{t-j}.$$

The kernel length K controls the receptive field. To model very long-range interactions, one needs large kernels, many layers, or dilation.

SSMs also lead to a convolutional representation, but with a kernel generated by the dynamical system:

$$K_j = C \bar{A}^j \bar{B}.$$

Thus SSMs behave like long convolutional filters whose structure is determined by $\bar{A}, \bar{B}, C$.

2.3 SSMs versus self-attention

Self-attention computes interactions between all pairs of positions. The classical attention mechanism uses

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} \right) V.$$

For a sequence of length T , the matrix $QK^\top$ is of size $T \times T$, so the complexity is typically

$$O(T^2).$$

SSMs avoid explicit pairwise comparisons. They compress past information into a state h_t and update it linearly:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t.$$

This gives $O(T)$ recurrent inference and, in S4-style models, efficient convolutional training.

2.4 SSMs versus Neural ODEs

Neural ODEs typically model

$$\dot{x}(t) = f_\theta(x(t)), \tag{4}$$

where f_θ is a nonlinear neural network. They learn nonlinear continuous-time flows directly on the observed or feature state.

SSMs instead use

$$\dot{h}(t) = Ah(t) + Bx(t). \tag{5}$$

The dynamics are linear in the hidden state $h(t)$, and the input enters as an external forcing term. This linear structure allows exact discretization, spectral analysis, and efficient convolutional computation.

3 ODE background: why matrix exponentials appear

Before discretizing SSMs, we need the ODE theory behind the solution.

3.1 Scalar homogeneous linear ODE

Consider

$$\dot{h}(t) = ah(t), \quad h(0) = h_0, \quad (6)$$

where $a \in \mathbb{R}$. This says that the rate of change of h is proportional to its current value.

We solve by separation of variables:

$$\frac{dh}{dt} = ah, \quad (7)$$

$$\frac{1}{h} dh = a dt, \quad (8)$$

$$\int \frac{1}{h} dh = \int a dt, \quad (9)$$

$$\log |h(t)| = at + c. \quad (10)$$

Exponentiating gives

$$h(t) = Ke^{at}. \quad (11)$$

Using $h(0) = h_0$ gives $K = h_0$. Therefore

$$\boxed{h(t) = e^{at}h_0.} \quad (12)$$

3.2 Matrix homogeneous linear ODE

Now let $h(t) \in \mathbb{R}^N$ and $A \in \mathbb{R}^{N \times N}$. Consider

$$\dot{h}(t) = Ah(t), \quad h(0) = h_0. \quad (13)$$

This is a coupled system. For example, if $N = 2$,

$$\frac{d}{dt} \begin{pmatrix} h_1(t) \\ h_2(t) \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} h_1(t) \\ h_2(t) \end{pmatrix}.$$

Thus

$$\dot{h}_1(t) = a_{11}h_1(t) + a_{12}h_2(t), \quad (14)$$

$$\dot{h}_2(t) = a_{21}h_1(t) + a_{22}h_2(t). \quad (15)$$

Each coordinate can depend on all coordinates.

The solution is

$$\boxed{h(t) = e^{At}h_0,} \quad (16)$$

where the matrix exponential is

$$\boxed{e^{At} = I + At + \frac{(At)^2}{2!} + \frac{(At)^3}{3!} + \dots} \quad (17)$$

3.3 Why does $e^{At}h_0$ solve the ODE?

Differentiate the matrix exponential term-by-term:

$$\frac{d}{dt}e^{At} = \frac{d}{dt} \left(I + At + \frac{A^2t^2}{2!} + \frac{A^3t^3}{3!} + \dots \right) \quad (18)$$

$$= A + A^2t + \frac{A^3t^2}{2!} + \dots \quad (19)$$

$$= A \left(I + At + \frac{A^2t^2}{2!} + \dots \right) \quad (20)$$

$$= Ae^{At}. \quad (21)$$

Therefore, if $h(t) = e^{At}h_0$, then

$$\dot{h}(t) = \frac{d}{dt} (e^{At}h_0) \quad (22)$$

$$= Ae^{At}h_0 \quad (23)$$

$$= Ah(t). \quad (24)$$

This proves that $e^{At}h_0$ is the solution.

3.4 How to form e^{At} in practice

The definition is the power series, but when A is diagonalizable,

$$A = V\Lambda V^{-1}, \quad (25)$$

where

$$\Lambda = \text{diag}(\lambda_1, \dots, \lambda_N), \quad (26)$$

then

$$e^{At} = Ve^{\Lambda t}V^{-1}, \quad (27)$$

with

$$e^{\Lambda t} = \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_N t}). \quad (28)$$

So the matrix exponential becomes scalar exponentials applied to the eigenvalues.

4 Solving the non-homogeneous linear ODE

The SSM contains an input term. We therefore need to solve

$$\dot{h}(t) = Ah(t) + g(t), \quad (29)$$

where in SSMs

$$g(t) = Bx(t). \quad (30)$$

This is a linear non-homogeneous ODE.

4.1 Derivation by integrating factor

Start from

$$\dot{h}(t) = Ah(t) + g(t). \quad (31)$$

Multiply both sides by e^{-At} :

$$e^{-At}\dot{h}(t) = e^{-At}Ah(t) + e^{-At}g(t). \quad (32)$$

The key is to recognize a product derivative. By the product rule,

$$\frac{d}{dt}(e^{-At}h(t)) = \frac{d}{dt}(e^{-At})h(t) + e^{-At}\dot{h}(t) \quad (33)$$

$$= -Ae^{-At}h(t) + e^{-At}\dot{h}(t). \quad (34)$$

Substitute $\dot{h}(t) = Ah(t) + g(t)$:

$$\frac{d}{dt}(e^{-At}h(t)) = -Ae^{-At}h(t) + e^{-At}(Ah(t) + g(t)) \quad (35)$$

$$= -Ae^{-At}h(t) + e^{-At}Ah(t) + e^{-At}g(t). \quad (36)$$

Since A commutes with e^{-At} ,

$$e^{-At}A = Ae^{-At}, \quad (37)$$

so the first two terms cancel:

$$\boxed{\frac{d}{dt}(e^{-At}h(t)) = e^{-At}g(t).} \quad (38)$$

Integrate from t_0 to t :

$$e^{-At}h(t) - e^{-At_0}h(t_0) = \int_{t_0}^t e^{-As}g(s) ds. \quad (39)$$

Multiply by e^{At} :

$$h(t) - e^{A(t-t_0)}h(t_0) = \int_{t_0}^t e^{A(t-s)}g(s) ds. \quad (40)$$

Therefore

$$\boxed{h(t) = e^{A(t-t_0)}h(t_0) + \int_{t_0}^t e^{A(t-s)}g(s) ds.} \quad (41)$$

This is the variation-of-constants formula.

4.2 Apply it to SSMs

For SSMs, $g(s) = Bx(s)$. Therefore

$$\boxed{h(t) = e^{A(t-t_0)}h(t_0) + \int_{t_0}^t e^{A(t-s)}Bx(s) ds.} \quad (42)$$

The interpretation is important:

- $e^{A(t-t_0)}h(t_0)$ is the old state propagated forward;
- the integral is the accumulated effect of all inputs between t_0 and t ;
- an input at time s is first injected through B , then propagated from s to t by $e^{A(t-s)}$.

5 Step 1: time discretization

We now derive the discrete SSM used in sequence models.

Let the sampling interval be $\Delta > 0$. We want to express $h(t)$ in terms of the previous sampled state $h(t - \Delta)$. Set

$$t_0 = t - \Delta. \quad (43)$$

Using (41),

$$h(t) = e^{A(t-(t-\Delta))}h(t - \Delta) + \int_{t-\Delta}^t e^{A(t-s)}Bx(s) \, ds \quad (44)$$

$$= e^{A\Delta}h(t - \Delta) + \int_{t-\Delta}^t e^{A(t-s)}Bx(s) \, ds. \quad (45)$$

Now perform the change of variable

$$u = t - s. \quad (46)$$

Then $s = t - u$ and $ds = -du$. When $s = t - \Delta$, $u = \Delta$. When $s = t$, $u = 0$. Therefore

$$\int_{t-\Delta}^t e^{A(t-s)}Bx(s) \, ds = \int_{\Delta}^0 e^{Au}Bx(t - u)(-du) \quad (47)$$

$$= \int_0^{\Delta} e^{Au}Bx(t - u) \, du. \quad (48)$$

Renaming u as s , we get the textbook ODE solution used in SSM discretization:

$$\boxed{h(t) = e^{A\Delta}h(t - \Delta) + \int_0^{\Delta} e^{As}Bx(t - s) \, ds.} \quad (49)$$

Key idea. Equation (49) says that the current state is the previous state evolved for time Δ plus the integrated influence of the input over the interval $[t - \Delta, t]$.

6 Discrete dynamical system: deriving $\bar{A}$, $\bar{B}$, and $\bar{C}$

In a real time series, we do not observe the full continuous function $x(s)$. We observe samples

$$x_1, x_2, \dots, x_T.$$

To convert the continuous system to a discrete system, we need an assumption on how $x(s)$ behaves between samples.

6.1 Zero-order hold assumption

The standard assumption used here is the zero-order hold assumption:

$$x(t - s) \approx x(t), \quad s \in [0, \Delta]. \quad (50)$$

That is, between two sample times, the input is treated as constant.

Starting from (49),

$$h(t) = e^{A\Delta}h(t - \Delta) + \int_0^\Delta e^{As}Bx(t - s) \, ds \quad (51)$$

$$\approx e^{A\Delta}h(t - \Delta) + \int_0^\Delta e^{As}Bx(t) \, ds. \quad (52)$$

Since B and $x(t)$ do not depend on s ,

$$h(t) = e^{A\Delta}h(t - \Delta) + \left(\int_0^\Delta e^{As} \, ds \right) Bx(t). \quad (53)$$

Define the discrete-time matrices

$$\boxed{\bar{A} = e^{A\Delta}}, \quad (54)$$

$$\boxed{\bar{B} = \left(\int_0^\Delta e^{As} \, ds \right) B}, \quad (55)$$

and usually

$$\boxed{\bar{C} = C, \quad \bar{D} = D.} \quad (56)$$

Then, writing $h(t) = h_t$, $h(t - \Delta) = h_{t-1}$, and $x(t) = x_t$, we obtain

$$\boxed{\begin{aligned} h_t &= \bar{A}h_{t-1} + \bar{B}x_t, \\ o_t &= \bar{C}h_t + \bar{D}x_t. \end{aligned}} \quad (57)$$

6.2 Derivation of $\bar{B} = A^{-1}(e^{A\Delta} - I)B$

We have

$$\bar{B} = \left(\int_0^\Delta e^{As} \, ds \right) B. \quad (58)$$

We want to evaluate the integral. Recall that

$$\frac{d}{ds}e^{As} = Ae^{As}. \quad (59)$$

If A is invertible, then

$$e^{As} = A^{-1} \frac{d}{ds}e^{As}. \quad (60)$$

Thus

$$\int_0^\Delta e^{As} \, ds = \int_0^\Delta A^{-1} \frac{d}{ds}e^{As} \, ds \quad (61)$$

$$= A^{-1} \int_0^\Delta \frac{d}{ds}e^{As} \, ds \quad (62)$$

$$= A^{-1} [e^{As}]_0^\Delta \quad (63)$$

$$= A^{-1} (e^{A\Delta} - e^{A0}) \quad (64)$$

$$= A^{-1} (e^{A\Delta} - I). \quad (65)$$

Therefore

$$\boxed{\bar{B} = A^{-1}(e^{A\Delta} - I)B.} \quad (66)$$

Remark 1. The expression (66) requires A to be invertible. The integral form

$$\bar{B} = \left(\int_0^\Delta e^{As} ds \right) B$$

is more general and remains valid even if A is singular.

6.3 What is the goal after discretization?

After discretization, the model is

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad o_t = \bar{C}h_t.$$

The goal is not merely to write a recurrence. The goal is to parametrize A, B, C so that the resulting discrete-time system has good properties:

- efficient computation for long T ;
- stable training;
- long memory;
- enough expressivity for time-series and sequence modeling.

7 Efficient computation: the convolutional trick

The recurrent update

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t$$

looks sequential, like an RNN. However, because it is linear and time-invariant, it can be rewritten as a convolution.

Assume $h_0 = 0$ and $\bar{D} = 0$. Then:

$$h_1 = \bar{B}x_1, \tag{67}$$

$$h_2 = \bar{A}h_1 + \bar{B}x_2 = \bar{A}\bar{B}x_1 + \bar{B}x_2, \tag{68}$$

$$h_3 = \bar{A}h_2 + \bar{B}x_3 = \bar{A}^2\bar{B}x_1 + \bar{A}\bar{B}x_2 + \bar{B}x_3, \tag{69}$$

$$h_4 = \bar{A}h_3 + \bar{B}x_4 = \bar{A}^3\bar{B}x_1 + \bar{A}^2\bar{B}x_2 + \bar{A}\bar{B}x_3 + \bar{B}x_4. \tag{70}$$

By induction,

$$h_t = \sum_{k=1}^t \bar{A}^{t-k} \bar{B}x_k. \tag{71}$$

The output is

$$o_t = \bar{C}h_t \tag{72}$$

$$= \sum_{k=1}^t \bar{C} \bar{A}^{t-k} \bar{B}x_k. \tag{73}$$

Define the SSM convolution kernel

$$K_j = \bar{C} \bar{A}^j \bar{B}, \quad j = 0, 1, 2, \dots \tag{74}$$

Then

$$o_t = \sum_{k=1}^t K_{t-k} x_k. \quad (75)$$

This is a causal convolution.

7.1 Interpretation of the kernel

The term

$$K_0 = \bar{C}\bar{B}$$

measures the effect of the current input x_t on the current output o_t .

The term

$$K_1 = \bar{C}\bar{A}\bar{B}$$

measures the effect of the previous input x_{t-1} on the current output.

The term

$$K_j = \bar{C}\bar{A}^j\bar{B}$$

measures the effect of an input from j steps ago.

So the SSM kernel is an impulse response: it tells us how the system reacts over time to a single input impulse.

7.2 Why this helps training

The recurrence is sequential:

$$h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_T.$$

This is efficient at inference time, but it is not maximally parallel during training.

The convolutional form computes

$$o = K * x.$$

Using fast convolution methods such as FFT, this can be computed efficiently and in parallel. This is one of the major reasons S4-style models are attractive: the same model has a recurrent view and a convolutional view.

Key idea. Training often uses the convolutional form for parallelism, while autoregressive inference uses the recurrent form for $O(1)$ state updates per time step.

8 Why we need a smart parametrization of A

The kernel contains powers

$$\bar{A}^j \bar{B}.$$

For long sequences, j can be very large. Therefore, we must avoid two problems:

- **exploding modes:** $\|\bar{A}^j\| \rightarrow \infty$;
- **vanishing modes:** $\|\bar{A}^j\| \rightarrow 0$ too quickly.

We want stability, but also long memory.

8.1 Eigendecomposition

A common strategy is to parametrize the continuous-time matrix A through its eigendecomposition:

$$\boxed{A = V\Lambda V^{-1}}. \quad (76)$$

Here:

- $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_N)$ contains the eigenvalues of A ;
- V contains the eigenvectors of A as columns;
- V^{-1} changes coordinates from the original basis to the eigenbasis;
- V changes coordinates from the eigenbasis back to the original basis.

Applying A to a vector can be interpreted as

$$Ah = V\Lambda V^{-1}h. \quad (77)$$

That means:

1. transform h into the eigenbasis using V^{-1} ;
2. scale/rotate each eigen-direction using Λ ;
3. transform back using V .

8.2 Why eigendecomposition makes powers and exponentials easy

If

$$A = V\Lambda V^{-1},$$

then

$$A^2 = (V\Lambda V^{-1})(V\Lambda V^{-1}) = V\Lambda^2 V^{-1}, \quad (78)$$

$$A^k = V\Lambda^k V^{-1}. \quad (79)$$

Similarly,

$$e^{A\Delta} = V e^{\Lambda\Delta} V^{-1}, \quad (80)$$

where

$$e^{\Lambda\Delta} = \text{diag}(e^{\lambda_1\Delta}, \dots, e^{\lambda_N\Delta}). \quad (81)$$

Therefore the eigenvalues of the discrete matrix $\bar{A} = e^{A\Delta}$ are

$$\boxed{\lambda_j(\bar{A}) = e^{\lambda_j(A)\Delta}}. \quad (82)$$

8.3 Stable complex eigenvalues

The parametrization used in the slide is

$$\boxed{\lambda_j(A) = -\alpha_j + i\omega_j, \quad \alpha_j > 0.} \quad (83)$$

This is deliberate. The real part is negative, and the imaginary part allows oscillations.

After discretization,

$$\lambda_j(\bar{A}) = e^{\lambda_j(A)\Delta} \quad (84)$$

$$= e^{(-\alpha_j + i\omega_j)\Delta} \quad (85)$$

$$= e^{-\alpha_j\Delta} e^{i\omega_j\Delta}. \quad (86)$$

The magnitude is

$$\boxed{|\lambda_j(\bar{A})| = e^{-\alpha_j\Delta} < 1.} \quad (87)$$

Thus the powers do not explode:

$$|\lambda_j(\bar{A})^k| = e^{-\alpha_j k\Delta}. \quad (88)$$

8.4 Memory interpretation

If α_j is large, then $e^{-\alpha_j\Delta}$ is much smaller than one. The mode decays quickly and only keeps short-term information.

If α_j is small, then

$$e^{-\alpha_j\Delta} \approx 1, \quad (89)$$

and therefore

$$e^{-\alpha_j k\Delta} \quad (90)$$

decays slowly. This mode can remember information over many time steps.

The imaginary part ω_j controls oscillatory behavior. The term

$$e^{i\omega_j\Delta}$$

is a complex rotation. Therefore each mode behaves like a damped oscillation:

$$e^{-\alpha_j k\Delta} e^{i\omega_j k\Delta}. \quad (91)$$

This is useful for time series with cycles, periodicity, and delayed effects.

8.5 Do we specify the eigenvalues or learn them?

In a neural SSM layer, we usually learn parameters that define the eigenvalues. For example, we learn unconstrained parameters $a_j \in \mathbb{R}$ and $\omega_j \in \mathbb{R}$, and set

$$\alpha_j = \text{softplus}(a_j) > 0. \quad (92)$$

Then

$$\lambda_j(A) = -\alpha_j + i\omega_j. \quad (93)$$

This guarantees stability while allowing the model to learn decay rates and frequencies.

8.6 What about V and V^{-1} ?

The full representation is

$$A = V\Lambda V^{-1}.$$

However, in many efficient SSM implementations, one works directly in the eigenbasis. Define

$$z_t = V^{-1}h_t. \quad (94)$$

Then, since $\bar{A} = Ve^{\Lambda\Delta}V^{-1}$,

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad (95)$$

$$V^{-1}h_t = e^{\Lambda\Delta}V^{-1}h_{t-1} + V^{-1}\bar{B}x_t, \quad (96)$$

$$z_t = e^{\Lambda\Delta}z_{t-1} + \tilde{B}x_t, \quad (97)$$

where

$$\tilde{B} = V^{-1}\bar{B}. \quad (98)$$

The output is

$$o_t = Ch_t = CVz_t = \tilde{C}z_t, \quad (99)$$

where

$$\tilde{C} = CV. \quad (100)$$

Thus the basis matrices V, V^{-1} can be absorbed into B and C . This is why simplified pseudo-code often uses a diagonal A directly.

9 Learning and gradients

A basic spectral SSM layer may learn

$$\theta = \{a, \omega, B, C, \Delta\}, \quad (101)$$

where

$$\alpha = \text{softplus}(a), \quad \lambda = -\alpha + i\omega. \quad (102)$$

The discrete parameters are

$$\bar{A}_j = e^{\lambda_j\Delta}, \quad (103)$$

$$\bar{B}_j = \frac{e^{\lambda_j\Delta} - 1}{\lambda_j} B_j, \quad (104)$$

where the second formula is the diagonal version of

$$\bar{B} = A^{-1}(e^{A\Delta} - I)B.$$

Given a training sequence $(x_1, \dots, x_T)$ and targets $(y_1^*, \dots, y_T^*)$, define predictions

$$\hat{y}_t = Ch_t, \quad (105)$$

and a loss, for example

$$\mathcal{L}(\theta) = \frac{1}{T} \sum_{t=1}^T \|\hat{y}_t - y_t^*\|^2. \quad (106)$$

Then parameters are updated by gradient descent or Adam:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta). \quad (107)$$

9.1 Example gradients

Let

$$\delta_t = \frac{\partial \mathcal{L}}{\partial \hat{y}_t}. \quad (108)$$

Since

$$\hat{y}_t = Ch_t, \quad (109)$$

the gradient with respect to C is

$$\frac{\partial \mathcal{L}}{\partial C} = \sum_{t=1}^T \delta_t h_t^\top. \quad (110)$$

The gradient with respect to B is more indirect because B influences all future hidden states. Automatic differentiation performs this through backpropagation through time or through the convolutional representation. Conceptually,

$$\frac{\partial \mathcal{L}}{\partial B} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial h_t} \frac{\partial h_t}{\partial B}. \quad (111)$$

For the spectral parameters, use the chain rule. Since

$$\bar{A}_j = e^{\lambda_j \Delta}, \quad (112)$$

we have

$$\frac{\partial \bar{A}_j}{\partial \lambda_j} = \Delta e^{\lambda_j \Delta} = \Delta \bar{A}_j. \quad (113)$$

Because

$$\lambda_j = -\alpha_j + i\omega_j, \quad (114)$$

then

$$\frac{\partial \lambda_j}{\partial \alpha_j} = -1, \quad \frac{\partial \lambda_j}{\partial \omega_j} = i. \quad (115)$$

Therefore

$$\frac{\partial \bar{A}_j}{\partial \alpha_j} = -\Delta e^{\lambda_j \Delta}, \quad (116)$$

$$\frac{\partial \bar{A}_j}{\partial \omega_j} = i\Delta e^{\lambda_j \Delta}. \quad (117)$$

If $\alpha_j = \text{softplus}(a_j)$, then

$$\frac{\partial \alpha_j}{\partial a_j} = \sigma(a_j), \quad (118)$$

where σ is the logistic sigmoid. Thus

$$\frac{\partial \mathcal{L}}{\partial a_j} = \frac{\partial \mathcal{L}}{\partial \alpha_j} \sigma(a_j). \quad (119)$$

10 Algorithm: training and inference for a spectral SSM layer

The following algorithm is written in an academic style. It combines both training and inference in one algorithm, while showing the parameters being learned.

Algorithm 1

Spectral State Space Model: Training and Inference

Input: training sequences $\{(x_{1:T}^{(m)}, y_{1:T}^{(m)})\}_{m=1}^M$, state size N , step size parameter Δ , learning rate η .

Parameters: raw decay $a \in \mathbb{R}^N$, frequency $\omega \in \mathbb{R}^N$, input matrix $B \in \mathbb{C}^{N \times d_{\text{in}}}$, readout matrix $C \in \mathbb{C}^{d_{\text{out}} \times N}$.

Initialization: initialize a, ω, B, C ; set $\alpha = \text{softplus}(a)$ and $\lambda = -\alpha + i\omega$.

Training phase:

for $q = 0, 1, \dots, Q - 1$ **do**

$$\alpha \leftarrow \text{softplus}(a), \quad \lambda \leftarrow -\alpha + i\omega,$$

$$\bar{A}_j \leftarrow \exp(\lambda_j \Delta), \quad j = 1, \dots, N,$$

$$\bar{B}_j \leftarrow \frac{\exp(\lambda_j \Delta) - 1}{\lambda_j} B_j,$$

compute predictions either recurrently:

$$h_0^{(m)} \leftarrow 0,$$

$$h_t^{(m)} \leftarrow \bar{A} \odot h_{t-1}^{(m)} + \bar{B} x_t^{(m)}, \quad t = 1, \dots, T,$$

$$\hat{y}_t^{(m)} \leftarrow \text{Re}(C h_t^{(m)}),$$

or convolutionally:

$$K_j \leftarrow C \bar{A}^j \bar{B}, \quad j = 0, \dots, T - 1,$$

$$\hat{y}_t^{(m)} \leftarrow \text{Re} \left(\sum_{k=1}^t K_{t-k} x_k^{(m)} \right),$$

$$\mathcal{L} \leftarrow \frac{1}{MT} \sum_{m=1}^M \sum_{t=1}^T \|\hat{y}_t^{(m)} - y_t^{(m)}\|^2,$$

$$(a, \omega, B, C) \leftarrow (a, \omega, B, C) - \eta \nabla_{a, \omega, B, C} \mathcal{L}.$$

Inference phase:

given $x_1, x_2, \dots, x_T$, set $h_0 = 0$,

for $t = 1, \dots, T$ **do**

$$h_t \leftarrow \bar{A} \odot h_{t-1} + \bar{B} x_t,$$

$$\hat{y}_t \leftarrow \text{Re}(C h_t).$$

return $\hat{y}_1, \dots, \hat{y}_T$.

Here $\odot$ denotes elementwise multiplication. This appears because the algorithm is written in the diagonal/eigenbasis representation, where $\bar{A}$ is represented by its diagonal entries.

11 Mamba and selective state-space models

Classical SSMs are linear time-invariant systems:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad o_t = \bar{C}h_t. \quad (120)$$

The same $\bar{A}, \bar{B}, \bar{C}$ are used at every time step. This gives efficient convolutional structure and long memory, but it also limits expressivity.

Mamba-style models introduce *selective* or input-dependent dynamics:

$$\boxed{h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad o_t = \bar{C}_t h_t.} \quad (121)$$

Now the transition can depend on the current input. A common form is

$$\bar{A}_t = \exp(\Delta_t A), \quad (122)$$

$$\bar{B}_t = (\exp(\Delta_t A) - I)(\Delta_t A)^{-1} \Delta_t B_t, \quad (123)$$

where Δ_t, B_t, C_t are functions of x_t .

11.1 Why input-dependence matters

In classical SSMs, the system has a fixed memory rule. In Mamba, the model can decide, depending on the input,

- what to write into memory;
- what to forget;
- what to expose to the output.

This resembles gates in LSTMs and GRUs, but it is built into an SSM-style architecture.

11.2 Why Mamba does not use the same convolutional trick

For linear time-invariant SSMs, the kernel

$$K_j = \bar{C} \bar{A}^j \bar{B}$$

does not depend on t . Therefore the whole layer can be written as one convolution.

In Mamba,

$$\bar{A}_t, \bar{B}_t, \bar{C}_t$$

change with t . Therefore the kernel is no longer time-invariant. The convolutional trick breaks.

Mamba instead uses a scan-based computation. The recurrence remains linear in the hidden state, so it can be implemented efficiently with specialized parallel scan kernels. This allows the model to keep approximately linear complexity in sequence length.

12 Summary of the full chapter

State Space Models start from a continuous-time dynamical system

$$\dot{h}(t) = Ah(t) + Bx(t), \quad o(t) = Ch(t) + Dx(t).$$

The homogeneous part $\dot{h} = Ah$ is solved by the matrix exponential

$$h(t) = e^{At}h(0).$$

The non-homogeneous system is solved by variation of constants:

$$h(t) = e^{A(t-t_0)}h(t_0) + \int_{t_0}^t e^{A(t-s)}Bx(s) \, ds.$$

Taking $t_0 = t - \Delta$ gives the textbook discretization formula

$$h(t) = e^{A\Delta}h(t - \Delta) + \int_0^\Delta e^{A\Delta-s}Bx(t-s) \, ds.$$

Assuming the input is constant between samples gives

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t,$$

where

$$\bar{A} = e^{A\Delta}, \quad \bar{B} = \left(\int_0^\Delta e^{A\Delta-s} \, ds \right) B.$$

If A is invertible,

$$\bar{B} = A^{-1}(e^{A\Delta} - I)B.$$

Unrolling the recurrence gives the convolution form

$$o_t = \sum_{k=1}^t K_{t-k}x_k, \quad K_j = \bar{C}\bar{A}^j\bar{B}.$$

This is the core computational trick behind S4-style models.

The main challenge is to parametrize A so that powers of $\bar{A}$ are stable and long-range. A spectral parametrization uses

$$A = V\Lambda V^{-1}, \quad \lambda_j(A) = -\alpha_j + i\omega_j, \quad \alpha_j > 0.$$

Then

$$\lambda_j(\bar{A}) = e^{-\alpha_j\Delta}e^{i\omega_j\Delta},$$

which gives stable damped oscillatory modes. Small α_j gives long memory.

Key idea. State Space Models are linear dynamical systems whose discretized impulse response becomes a learnable long convolution kernel. S4 emphasizes efficient long-memory convolution; Mamba adds input-dependent selectivity to improve expressivity while keeping near-linear complexity.